



MERRIMACK COLLEGE

CSC 6301

Week 6

Version Control and Issue Tracking

Software Design & Documentation – GK Gautham Krishnamurthy

Last week feedback

Are we recording?

What we really wanted to track?

- Reuse what we did in assignment 4 in assignment 5. Did you lose the original implementation?
 - If you were to change the implementation to use a web push, will it work quickly?
 - Why reuse code? (do not reinvent the wheel!). It needs to be correct! How to minimize changes? Why minimize?
 - When we code with SOLID guidelines, reusing code becomes easy. Please see the note on design patterns in the content section of this module
 - What should code documentation (using pydocs or javadocs) convey? It's not enough that you have Javadoc. What does it convey?
 - Do NOT assume the next person knows the context and tech details. Is it professional?
 - Running a debugger...
-



MERRIMACK COLLEGE

Presentation Agenda

Week 6

- Version Control
 - a. Usual Numbering
 - i. Versions, Subversions, and Patches
 - b. Environments and Compatibility
 - c. Version Documentation
- Issue Tracker
 - a. Collaborative Projects
 - b. Issue Tracker Tools
 - i. GitHub Repository
- This Week's tasks



MERRIMACK COLLEGE

CSC 6301
Software Design &
Documentation

Week 6
Version
Control
and
Issue
Tracking



Version Control

Every software development endeavour will have versions, even those meant to be just a single one time code. Hence, the control of versions is an integral part of the software development.

This topic is discussed in terms of the usual steps:

- Usual Numbering;
- Environments and Compatibility;
- Version Documentation.

*"To improve is to change;
To be perfect is to change often."*
Winston Churchill



Usual Numbering

The maintenance phase is the more extensive one in SDLC and the outcomes in this phase are summarized in new releases.

Usually, software products are numbered in up to three numbers.

- Version M.N.b
 - M - major release
 - N - minor release
 - b - build



MERRIMACK COLLEGE

Video: [Software Versioning Numbers](#).

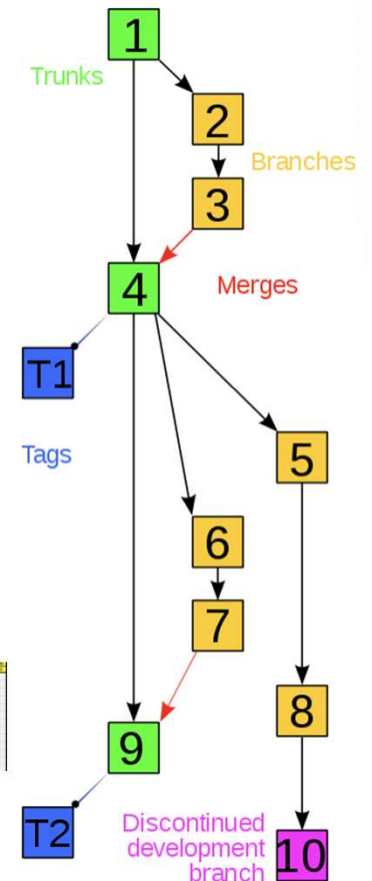
Version Control

The numbering discipline is enough for software public clients, but for software developers keep track of all changes it is necessary to control the produced versions.

Version control systems are responsible for **managing source code** revisions that usually become new software versions:

- not all revisions make it out to public software versions;
- version control can also be used to manage versions of software documentation.

Usually, the revisions can be described by a DAG directed acyclic graph structure with a main sequence (trunk) with subversions splits (branches) and joins (merges).



Environments and Compatibility

While reusing software it is usual to choose the latest version of the package, unless you have a reason for not doing it:

- Familiarity
 - you have used it before;
 - the documentation is better;
- Compatibility
 - the versions have different characteristics (functions, parameters, complexity, etc).



Your software with the reused code will rely on the versions used when you build it.

It is necessary to wrap-up the version number of each package used, this is called the development **environment**.

Some development environments encapsulate the versions of each package to be used to build up new versions of your software.



Environments and Compatibility in Python

The use of **pip** command to install Python packages is an effective tool to set up environments.

```
Collecting absl-py==0.13.0
  Downloading absl_py-0.13.0-py3-none-any.whl (132 kB)
    | 132 kB 8.2 MB/s
Requirement already satisfied: astor==0.8.1 in /usr/local/lib/python3.7/site-packages (from absl-py==0.13.0)
Requirement already satisfied: cached-property==1.5.2 in /usr/local/lib/python3.7/site-packages (from astor==0.8.1)
Collecting cachetools==4.2.2
  Downloading cachetools-4.2.2-py3-none-any.whl (11 kB)
Collecting certifi==2021.5.30
  Downloading certifi-2021.5.30-py2.py3-none-any.whl (145 kB)
    | 145 kB 65.1 MB/s
Collecting charset-normalizer==2.0.3
  Downloading charset_normalizer-2.0.3-py3-none-any.whl (35 kB)
Collecting colorama==0.4.4
  Downloading colorama-0.4.4-py2.py3-none-any.whl (16 kB)
Collecting conllu==4.4
  Downloading conllu-4.4-py2.py3-none-any.whl (15 kB)
Collecting emoji==1.6.1
  Downloading emoji-1.6.1.tar.gz (170 kB)
    | 170 kB 69.3 MB/s
Collecting gast==0.2.2
  Downloading gast-0.2.2.tar.gz (10 kB)
```

The use of **pip** (preferred installer program) command to install Python packages is an effective tool to set up environments.

pip -r install requirements.txt

Will install the packages listed in a file using their respective informed versions, for example:

```
absl-py==0.13.0
astor==0.8.1
cachetools==4.2.2
certifi==2021.5.30
charset-normalizer==2.0.3
colorama==0.4.4
emoji==1.6.1
google-auth==1.33.1
numpy==1.18.5
```



MERRIMACK COLLEGE

Video: [Python Basics Tutorial Pip Install Multiple Modules With Requirements File.](#)

Version Documentation

Full Version Documentation

- Every time a full version is finished and made public the documentation should state:
 - the changes from the previous version, notably:
 - the requirements changes;
 - new and deprecated features (classes and functions, etc);
 - bugs addressed.

Code Revision Documentation

- Every patch or working revision must state:
 - to whom it is addressed (public, developers, testers, beta testers, etc.);
 - the implementation issues;
 - and every full version documentation items..



Version documentation also refers to managing versions of documents

Issue Tracker

Some software problems do not have simple solutions, as such, those issues must be dealt periodically.

This is particularly important to open software projects (as most sources for software reuse).

This is addressed in the following topics:

- Collaborative Projects;
- Issue Tracker Tools.

"Remember!"

Spock at the end of "The Wrath of Khan" (1982)



Collaborative Projects

Collaborative Software Projects

- Proprietary projects split in several teams
 - Global Software Engineering
 - Coordinated/Specialized teams
 - Follow the Sun methodology
 - Real-time collaboration
- Open Software collaborations
 - Board managed endeavors
 - Linux Operating System
 - Crowdsourcing
 - GIMPS - Great Internet Mersenne Prime Search
 - Kaggle - an online community of machine learning practitioners



Project Management
Collaboration Tools

Text: [The 12 Best PMCT](#)



kaggle



MERRIMACK COLLEGE

Wikipedia: [Collaborative Project Management](#).

Issue Tracking

From Bug Tracker to Issue Tracker

- Historically, software companies kept track of perceived flaws (bugs) in their produced software considering future maintenance;
 - This was done in a bug tracker system, but eventually, bug tracker systems evolve to issue tracker, as not all perceived flaws were actually bugs.
- Collaborative projects employ issue tracker systems as a tool to both keep track and collaboratively discuss how to deal with each issue:
 - fixing the bug;
 - finding a way to avoid the issue;
 - documenting the issue;
 - collecting data about the issue.



Issue Tracker Approaches

Issue Tracking Practices

- Some collaborative software development, as open software, make them available at repositories together with blog-like issue tracker entries:
 - This is a common, for example, at GitHub repository;
 - Such practices are effective as open software users and developers are very interested in identifying issues, but it is an *ad hoc* procedure.

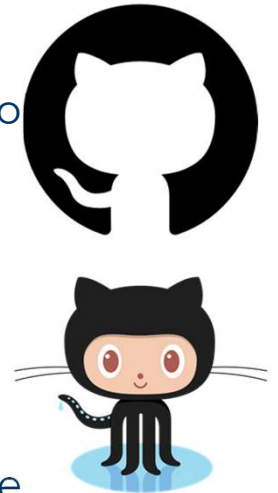
Issue Tracking Tools

- More centralized projects use specialized software to keep track of issues both at coding, testing, and maintenance SDLC steps:
 - Such systems provide a controlled approach and systematic registry of issues and outcomes.



GitHub Repository

- An Internet repository for software development providing version control and largely used by developers:
 - 83 million developers;
 - 200 million repositories (28 million public);
- GitHub was acquired by Microsoft in 2018, but it remained working with the free using mode (registration required to store, not to download);
- Users can define how their projects will be stored and distributed:
 - It is not required, but documentation and issue tracking systems are often standardized by GitHub users.



**Build on what's
been built**

Wikipedia: [GitHub](#) - Official Page: github.com.



MERRIMACK COLLEGE

Video: [Learn Git In 15 Minutes](#).

GitHub Project Handover Checklist

Ensuring Maintainability and "Ownership" Readiness

To allow another professional to successfully take over or contribute to your repository, include these essential components:

Comprehensive README.md

- **Project Purpose:** A high-level summary of what the software does.
- **Quick Start:** Clear, step-by-step instructions to get the app running in under 5 minutes.
- **Prerequisites:** List of required OS, runtimes (Java JDK 17+, Python 3.10+), or hardware.

Dependency Management

- A functional manifest file (`pom.xml` for Maven, `build.gradle` for Gradle, or `requirements.txt` for Python).
- *Tip:* A developer should be able to run one command (e.g., `pip install -r requirements.txt`) to set up the environment.

Documentation (API & Logic)

- **Inline Docs:** Ensure Javadoc or Pydoc is present for all public methods and classes.
- **Architecture Overview:** A brief description of the design patterns used (e.g., "Uses the Observer pattern for event handling").

Testing Suite

- Include a `tests/` directory with Unit Tests (JUnit/PyTest).
- Provide the specific command used to run tests and verify the build's health.

Environment Configuration

- A `.gitignore` file to prevent sensitive data (API keys) or IDE-specific files (`.vscode`, `.idea`) from cluttering the repo.
- An `.env.example` file if the project requires configuration variables.

Licensing

- A `LICENSE` file (e.g., MIT, Apache 2.0) so the professional knows their legal rights to modify and distribute the code.

"Ownership" means the new developer shouldn't have to email the original author to ask how to build the project. If the repository is truly professional, the code and documentation should speak for themselves.



Example of unit testing in Java (Calculator class)

```
public class Calculator {  
    public static int add(int a, int b) {  
        return a + b;  
    }  
  
    public static int subtract(int a, int b) {  
        return a - b;  
    }  
  
    public static int multiply(int a, int b) {  
        return a * b;  
    }  
  
    public static int divide(int a, int b) throws ArithmeticException {  
        if (b == 0) {  
            throw new ArithmeticException("Division by zero");  
        }  
        return a / b;  
    }  
}
```



CalculatorTest Class

```
public class CalculatorTest {
    public static void main(String[] args) {
        CalculatorTest test = new CalculatorTest();
        test.testAdd();
        test.testSubtract();
        test.testMultiply();
        test.testDivide();
        test.testDivideByZero();
    }

    public void testAdd() {
        Calculator calculator = new Calculator();
        int result = calculator.add(2, 3);
        assert result == 5 : "Test Add Failed: Expected 5 but got " + result;
        System.out.println("Test Add Passed");
    }

    public void testSubtract() {
        Calculator calculator = new Calculator();
        int result = calculator.subtract(5, 3);
        assert result == 2 : "Test Subtract Failed: Expected 2 but got " + result;
        System.out.println("Test Subtract Passed");
    }

    public void testMultiply() {
        Calculator calculator = new Calculator();
        int result = calculator.multiply(2, 3);
        assert result == 6 : "Test Multiply Failed: Expected 6 but got " + result;
        System.out.println("Test Multiply Passed");
    }
}
```

```
public void testDivide() {
    Calculator calculator = new Calculator();
    try {
        int result = calculator.divide(6, 3);
        assert result == 2 : "Test Divide Failed: Expected 2 but got " + result;
        System.out.println("Test Divide Passed");
    } catch (ArithmeticException e) {
        System.out.println("Test Divide Failed with Exception: " + e.getMessage());
    }
}

public void testDivideByZero() {
    Calculator calculator = new Calculator();
    try {
        calculator.divide(1, 0);
        System.out.println("Test Divide By Zero Failed: Expected ArithmeticException");
    } catch (ArithmeticException e) {
        assert "Division by zero".equals(e.getMessage()) : "Test Divide By Zero Failed: Expected 'Division by zero' but got " + e.getMessage();
        System.out.println("Test Divide By Zero Passed");
    }
}
}
```



Explanation

- **testAdd()**: Tests the add method of the Calculator class.
- **testSubtract()**: Tests the subtract method of the Calculator class.
- **testMultiply()**: Tests the multiply method of the Calculator class.
- **testDivide()**: Tests the divide method of the Calculator class.
- **testDivideByZero()**: Tests the divide method for division by zero, ensuring it throws an ArithmeticException.



Now I am using what I developed

```
import java.util.Scanner;

public class WorldsBestApp{
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter two numbers : ");
        int num1 = scanner.nextInt();
        int num2 = scanner.nextInt();
        int thesum=0;
        scanner.close();
        thesum = Calculator.add(num1, num2);
        System.out.println("The sum is " + thesum);
    }
}
```



Running the tests

```
gk-mac:Module6 $ javac -d bin Calculator.java
gk-mac:Module6 $ java -cp bin -ea CalculatorTest.java
Test Add Passed
Test Subtract Passed
Test Multiply Passed
Test Divide Passed
Test Divide By Zero Passed
gk-mac:Module6 $ java -cp bin WorldsBestApp.java
Enter two numbers : 3
4
The sum is 7
gk-mac:Module6 $
```

-ea is to enable assertions!



MERRIMACK COLLEGE

Code Coverage

If all tests pass, you will see corresponding "Test Passed" messages. If any test fails, an assertion error will be thrown with a detailed message.

Coverage

This approach gives good code coverage as it tests:

- Basic functionality of the arithmetic operations.
- Edge cases, such as division by zero.

Using assertions and manual testing methods, you can cover a wide range of scenarios without relying on external libraries.



Code Static Analysis

Static analysis of code involves *examining the source code for*

- potential errors,
- code smells, and
- adherence to coding standards

without actually executing the program. This can be achieved using various tools and techniques.



1. Choose a Static Analysis Tool

There are several popular tools for static analysis of (Java) code:

- **Checkstyle**: Focuses on coding standards and style.
- **PMD**: Detects code smells, potential bugs, and code style issues.
- **FindBugs (SpotBugs)**: Identifies potential bugs in Java programs.
- **SonarQube**: A comprehensive platform that integrates with various tools to provide detailed analysis.



2. Install the Tool (Checkstyle)

Via Maven:

Add the Checkstyle plugin to your pom.xml:

```
<project>
...
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-checkstyle-plugin</artifactId>
      <version>3.1.2</version>
      <configuration>
        <configLocation>google_checks.xml</configLocation>
      </configuration>
      <executions>
        <execution>
          <phase>validate</phase>
          <goals>
            <goal>check</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
</project>
```

Via Gradle:

Add the Checkstyle plugin to your build.gradle:

```
plugins {
    id 'checkstyle'
}

checkstyle {
    toolVersion = '8.36.2'
    config =
        resources.text.fromFile('config/checkstyle/checkstyle.xml')
}

tasks.withType(Checkstyle) {
    reports {
        xml.required = false
        html.enabled = true
    }
}
```



2. Install the Tool (PMD)

Via Maven:

Add the PMD plugin to your pom.xml

```
<project>
  ...
  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-pmd-plugin</artifactId>
        <version>3.13.0</version>
        <executions>
          <execution>
            <phase>validate</phase>
            <goals>
              <goal>pmd</goal>
              <goal>cpd-check</goal>
            </goals>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>
</project>
```

Via Gradle:

Add the PMD plugin to your build.gradle:

```
plugins {
    id 'pmd'
}

pmd {
    toolVersion = '6.21.0'
    ruleSets = ['category/java/bestpractices.xml']
}

tasks.withType(Pmd) {
    reports {
        xml.required = false
        html.enabled = true
    }
}
```



2. Install the Tool (FindBugs - SpotBugs)

Via Maven:

Add the SpotBugs plugin to your pom.xml

```
<project>
  ...
  <build>
    <plugins>
      <plugin>
        <groupId>com.github.spotbugs</groupId>
        <artifactId>spotbugs-maven-plugin</artifactId>
        <version>4.0.0</version>
        <executions>
          <execution>
            <goals>
              <goal>check</goal>
            </goals>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>
</project>
```

Via Gradle:

Add the SpotBugs plugin to your build.gradle:

```
plugins {
    id 'com.github.spotbugs' version '4.0.0'
}

spotbugs {
    toolVersion = '4.0.0'
    ignoreFailures = false
    effort = 'max'
    reportLevel = 'high'
}

tasks.withType(SpotBugsTask) {
    reports {
        xml.required = false
        html.enabled = true
    }
}
```



3. Configure the tool and 4. Run the static analysis

3. Configure the Tool

Each tool requires configuration files that define the rules and checks to be applied. These configuration files can be customized to meet the specific needs of your project. Most tools provide default configurations that can be used as a starting point.

4. Run the Static Analysis

After configuring the tool, run the analysis using your build tool or directly via the command line. The tool will generate reports highlighting issues in the code.



5. Review and Fix Issues

Review the generated reports, prioritize the issues, and make necessary code changes to address the detected problems.

This step often involves:

- Fixing potential bugs.
- Refactoring code to eliminate code smells.
- Ensuring the code adheres to coding standards.



6. Integrate with CI/CD Pipeline

Integrate static analysis tools into your CI/CD pipeline to ensure continuous code quality checks. This ensures that any new code changes are automatically analyzed for potential issues before merging.

Conclusion

Static analysis is a crucial practice in software development to **ensure code quality** and **maintainability**. By choosing the right tools, configuring them appropriately, and integrating them into your development workflow, you can catch potential issues early and maintain high standards for your codebase.



Behavioral patterns deal with object communication

Some examples include:-

- Chain of responsibility,
- Command,
- Interpreter,
- Iterator,
- Mediator,
- Memento,
- **Observer**,
- State and
- Visitor patterns.



What is Observer Pattern?

- This is a behavioral design pattern.
- Observer pattern is used to solve the problem of notifying multiple objects of a change to keep them in sync like the Model-View-Controller concept.
- Useful for event management kind of scenarios.
- Two classes are involved.
- The Observable class is where the actual data change is occurring. It has information about all the interested objects that need to be notified.
- The Observer is typically an abstract class providing the interface to which concrete classes interested in the events need to be compliant to.



Example of implementation of the Observer design pattern in Java

The Observer design pattern is a behavioral design pattern in which an object, called the subject, maintains a list of its dependents, called observers, and notifies them of any state changes, usually by calling one of their methods. Here's an example of how you can implement the Observer design pattern in Java:

Step 1: Define the Observer Interface

```
interface Observer {  
    void update(String message);  
}
```



Observer design pattern – step 2

Step 2: Define the Subject Class

```
import java.util.ArrayList;
import java.util.List;

class Subject {
    private List<Observer> observers = new ArrayList<>();
    private String message;

    public void addObserver(Observer observer) {
        observers.add(observer);
    }

    public void removeObserver(Observer observer) {
        observers.remove(observer);
    }
}
```

```
    public void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(message);
        }
    }

    public void setMessage(String message) {
        this.message = message;
        notifyObservers();
    }
}
```



Observer design pattern – step 3

Step 3: Create Concrete Observer Classes

```
class ConcreteObserverA implements Observer {  
    private String name;  
  
    public ConcreteObserverA(String name) {  
        this.name = name;  
    }  
  
    @Override  
    public void update(String message) {  
        System.out.println("Observer " + name + " received message: " + message);  
    }  
}
```

```
class ConcreteObserverB implements Observer {  
    private String name;  
  
    public ConcreteObserverB(String name) {  
        this.name = name;  
    }  
  
    @Override  
    public void update(String message) {  
        System.out.println("Observer " + name + " received message: " + message);  
    }  
}
```



Observer design pattern – step 4

Step 4: Test the Observer Pattern

```
public class ObserverPatternDemo {  
    public static void main(String[] args) {  
        Subject subject = new Subject();  
  
        Observer observerA = new ConcreteObserverA("A");  
        Observer observerB = new ConcreteObserverB("B");  
  
        subject.addObserver(observerA);  
        subject.addObserver(observerB);  
  
        subject.setMessage("Hello, Observers!");  
        subject.setMessage("Another message!");  
    }  
}
```



Observer design pattern – Explanation

Explanation

1.Observer Interface: Defines the update method that observers must implement.

2.Subject Class: Maintains a list of observers and notifies them of changes. It has methods to add, remove, and notify observers.

3.Concrete Observer Classes: Implement the Observer interface and define how they respond to updates.

4.Test the Pattern: In the ObserverPatternDemo class, we create a Subject and attach Observer instances to it. When the subject's state changes (via the setMessage method), it notifies all observers.

When you run the ObserverPatternDemo class, you will see the output showing that both observers receive updates when the subject's state changes.



Observer design pattern – Examples

The **Observer Design Pattern** is widely used in software development, particularly when an object (the "subject") needs to notify other objects (the "observers") about changes in its state without being tightly coupled to them.

Here are some common examples where the observer pattern is implemented:



Observer design pattern – 1. UI

User Interface (UI) Event Handling

- **Example:**

In GUI frameworks like Java Swing or .NET's WinForms, the observer pattern is used to notify components about user actions like button clicks or text input. When a user interacts with the UI, events like mouse clicks, keystrokes, or selections trigger notifications to registered listeners (observers).

- **Use case:**

Buttons, text fields, or checkboxes notify multiple listeners when their state changes.



Observer design pattern – 2. Model-View-Controller (MVC) Architecture

With Django, you can take web applications from concept to launch in a matter of hours. Django takes care of much of the hassle of web development, so you can focus on writing your app without needing to reinvent the wheel. It's free and open source.

Ruby on Rails (<https://rubyonrails.org>)

A web-app framework that includes everything needed to create database-backed web applications according to the Model-View-Controller (MVC) pattern.

Model-View-Controller (MVC) Architecture

- **Example:**

In MVC frameworks like Django or Ruby on Rails, the Observer pattern can be seen in how the **View** updates when changes in the **Model** occur.

- **Use case:**

When the model's data changes, all views that depend on this data are automatically notified to update their display, keeping the user interface in sync with the underlying data.



Observer design pattern – 3. Notification Systems

Notification Systems

Example:

Email, SMS, or app notifications systems use the observer pattern to notify users of updates.

Use case:

In a social media application, when a user posts new content, the system notifies all followers of that user through various channels like email, push notifications, or in-app messages.



Observer design pattern – 4. Data Streaming

Real-time Data Streaming

Example:

In stock trading platforms, weather apps, or live sports updates, observer patterns are used to push updates to multiple clients.

Use case:

A stock price tracker uses the observer pattern to notify all clients (observers) when stock prices change. Any changes are propagated to all the views, ensuring users see the latest prices in real-time.



Observer design pattern – 5. Distributed Systems

Distributed Event-Driven Systems

Example:

Event bus systems like **Apache Kafka** or **Google Cloud Pub/Sub** implement the observer pattern for event streaming.

Use case:

Microservices architecture where services subscribe to events. When an event is published (such as a new order in an e-commerce app), all interested services (like billing, inventory, shipping) are notified and can react accordingly.



Observer design pattern – 6. Document view

Document-View Architecture

Example:

Microsoft Word, Excel, or Google Docs use the observer pattern in their document-view models.

Use case:

Multiple views (different windows showing the same document) are synchronized. When the document changes, all views are updated instantly, allowing real-time collaboration or document consistency.



Observer design pattern – 7. Game dev

Observer in Game Development

Example:

In game engines like **Unity** or **Unreal Engine**, the observer pattern is used to manage game events.

Use case:

In a multiplayer game, when the game state changes (e.g., a player scores a point), all connected clients (observers) are notified so that the game screen reflects the updated score or action in real-time.

Observer design pattern – 8. Logging

Logging Frameworks

Example:

Logging libraries like **Log4j** or **SLF4J** use the observer pattern to notify multiple logging handlers (observers) about log messages generated by different parts of the application.

Use case:

When a log message is created, it is sent to all log listeners, such as file loggers, console loggers, and external monitoring systems.



Observer design pattern – 9. Database Systems (Triggers)

Database Systems (Triggers)

Example:

In databases, the observer pattern is analogous to database triggers.

Use case:

When a record in a database table changes (insert, update, delete), the database triggers notify other systems or parts of the application to perform actions, such as updating a cache or triggering business logic.



Observer design pattern – 10. Messaging Apps

Observer in Messaging Apps

Example:

In messaging applications like Slack or WhatsApp, when a user sends a message, all recipients (observers) are notified instantly.

Use case:

Real-time messaging where the message delivery service notifies multiple participants (observers) of a new message or status change (e.g., when someone reads the message).

Observer design pattern – Summary

Summary

The Observer pattern is invaluable in scenarios where:

- You need loose coupling between objects.
- Multiple objects need to be notified of state changes in one object.
- A subject can have many observers, with observers dynamically added or removed at runtime.



This Week's tasks

- In-class exercises
- Programming project #6

Tasks

- Create a profile on GitHub;
- Create a repository for your last two programming projects (Projects #4 and #5) in your GitHub profile.
- Make sure it can be accessed!



In-Class Exercise - Week 6

Create your profile in GitHub using your Merrimack credentials.

- Your profile doesn't need to be particularly sophisticated, just an ordinary developer profile capable of creating a project and uploading project files;
- Once a new user can access your Project, can a reasonably competent software professional use it and own it?

Your task:

- Create your profile by Friday and publish it in the discussion forum;
- Feel free to visit the profile of your colleagues as posted in the discussion forum.

This discussion posts counts towards the In-Class Exercises grade.



MERRIMACK COLLEGE

Progr. Project #6 - Week 6 - Assignment 01

- Pack your Project #4 and your Project #5 and upload them in your created GitHub profile;
- Your projects should approximatively follow the recommendations of the GitHub with respect to documentation.

Your task:

- Submit in Canvas the link of your projects so I could access it.



MERRIMACK COLLEGE

This assignment counts towards the Coding Projects grade.

What we covered today

Version Control, Documentation, and Collaboration

Version Control Foundations

- The "Why":** Managing change is an integral part of software development.
- Structure:** Revisions are often managed using a Directed Acyclic Graph (DAG) with trunks, branches, and merges.
- Standard Numbering (M.N.b):** Major release (M), Minor release (N), and Build (b).

Environments and Compatibility

- Dependency Management:** The importance of wrapping up package versions to create a stable development environment.
- Python Tooling:** Using pip and requirements.txt to install specific package versions (e.g., pip -r install requirements.txt).

Documentation and Maintenance

- Full Version Documentation:** Must state changes from previous versions, including requirement changes, new/deprecated features, and addressed bugs.
- "Owner" Mindset:** Documentation should be clear enough that the next person (a "reasonably competent professional") can understand and own the project without extra help.

Issue Tracking and Collaboration

- From Bugs to Issues:** Systems have evolved from simple bug trackers to comprehensive issue trackers to handle all software flaws and collaborative discussions.
- Tools:** Overview of GitHub as a repository and the use of centralized tools like Google Issue Tracker.

Design Patterns & Quality

- Observer Pattern:** A behavioral pattern used to notify multiple objects of state changes (e.g., UI events, MVC architecture).
- Static Analysis:** Using tools like Checkstyle, PMD, and SpotBugs to find potential errors and "code smells" without executing the program.

Why use design patterns in your projects?

- ✓ Using these patterns makes your code compliant with **SOLID guidelines**. They make the code easier to maintain and extend by "reasonably competent professionals".



” **Check out recommended texts and videos**

- **Make sure your Week 5 tasks are done**
- **Do the In-class exercise by next Friday**
- **Do the coding project by next Tuesday**

Next Week - Software Debugging, Testing, and Profiling



MERRIMACK COLLEGE